

Cours 3 PAM

Révisions :

- **Composable** : Pour l'interface graphique
 - Fonction annotée par **@Composable**
 - Exemples de méthodes Composables
 - Box, Row, Column, **Card** (Pour afficher en petites vignettes)
 - -> Organisation/Placement avec ces conteneurs
 - Text, Image -> Affiche du contenu (texte, image)
 - Spacer -> Espacement
 - Button -> Evènement/Interaction
 - **Modifier** : Objet qui regroupe un ensemble d'éléments de décoration
 - Padding : Positionnement/Ecartement
 - FillMaxSize ou FillMaxWidth
 - Weight : Utile que dans le cadre des rows et columns pour répartir l'espace
 - Bg : Changer le background
 - FontSize
 - FontStyle
 - **Preview vs Application** : Permet de prévisualiser notre application sans avoir à l'exécuter
 - Utile d'avoir une fonction **@Composable** commune entre ces deux éléments
 - **State** : Nos éléments composables ne sont pas faits pour « mémoriser » des informations
 - On va utiliser donc des variables de type **remember** qui va conserver les variables entre deux recompositions (Réaffichage, modification de variables)
 - **DataSource**
 - Classe qui stocke des données
 - **Var vs Val**
 - Une val est une valeur fixe qui ne peut pas être modifiée
 - Une Var peut être modifiée
-

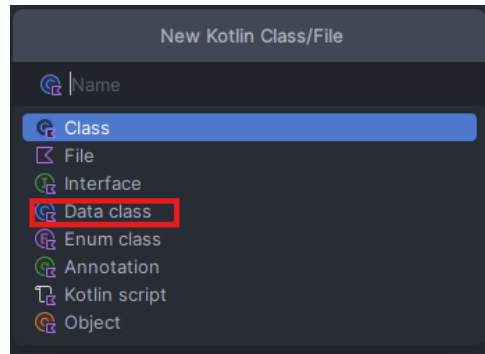
Construire une liste scrollable :

On va utiliser une **data class**, c'est ici que l'on initialise notre liste d'items qui va rassembler le nom et le types de nos éléments.

Package : Organisation des fichiers dans un dossier pour organiser nos classes.

MVC : Séparer le code de l'affichage des données.

On va créer une classe dans notre **package** 'model' :



La classe va être nommée « Affirmation »

Voici notre classe affirmation :

```
data class Affirmation(  
    @StringRes val stringResourceId: Int,  
    @DrawableRes val imageResourceId: Int  
)
```

@StringRes et @DrawableRes sont utilisés par Kotlin lors de l'exécution.

Dans `datasource.kt` dans le package **data**, on va pouvoir y initialiser des instances de notre classe **affirmation**.

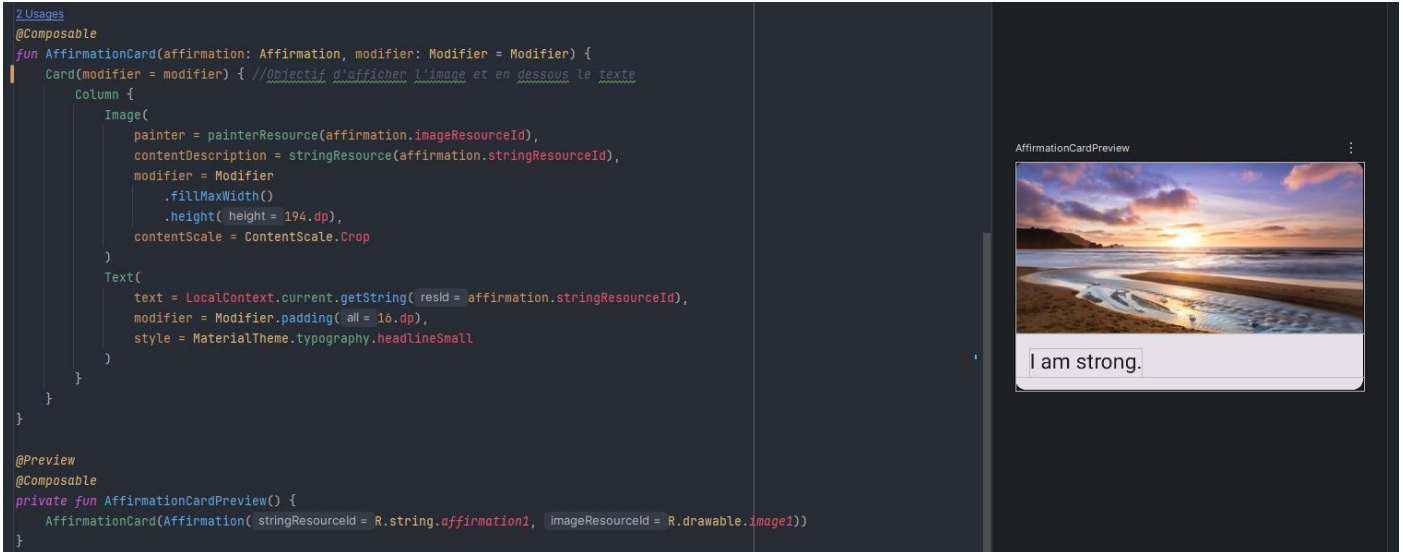
```
4 class Datasource() {  
5     1 Usage  
6     fun loadAffirmations(): List<Affirmation> {  
7         return listOf<Affirmation>(  
8             Affirmation("I am strong.", imageResourceId = R.drawable.image1),  
9             Affirmation("I believe in myself.", imageResourceId = R.drawable.image2),  
10            Affirmation(stringResourceId = R.string.affirmation3, imageResourceId = R.drawable.image3),  
11            Affirmation("Every challenge in my life is an opportunity to learn f...", imageResourceId = R.drawable.image4),  
12            Affirmation("I have so much to be grateful for.", imageResourceId = R.drawable.image5),  
13            Affirmation("Good things are always coming into my life.", imageResourceId = R.drawable.image6),  
14            Affirmation("New opportunities await me at every turn.", imageResourceId = R.drawable.image7),  
15            Affirmation("I have the courage to follow my heart.", imageResourceId = R.drawable.image8),  
16            Affirmation("Things will unfold at precisely the right time.", imageResourceId = R.drawable.image9),  
17            Affirmation("I will be present in all the moments that this day brings.", imageResourceId = R.drawable.image10))  
18        }  
19    }  
20 }
```

LoadAffirmation va générer une liste d'Affirmations

Avec tout ça, j'ai en ma possession une fonction qui va me permettre de récupérer des données (images, texte) que je vais pouvoir afficher sur mon application.

Objectif : Etant donné une Affirmation, pouvoir afficher notre élément

➔ On va utiliser la méthode **Card**



On a notre fonction d'affichage de Card

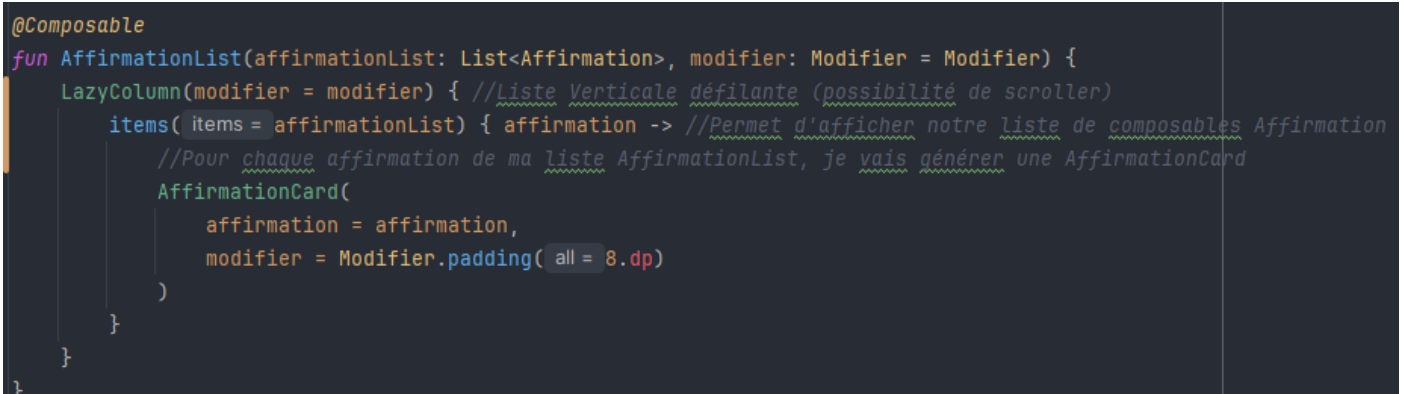
On va, à travers la fonction `AffirmationList`, réunir plusieurs informations

Pour afficher une liste verticale scrollable, on va utiliser la méthode **LazyColumn**. (tous les éléments ne seront pas chargés en même temps mais uniquement ceux que l'on veut afficher)

➔ Propose le scrolling par défaut

Dans **LazyColumn**, on va avoir une liste de composables, on va donc utiliser **Items** va permettre de générer un ensemble de composables.

NB : Dans notre **@Preview**, on peut mettre dans ses paramètres **device** afin de choisir nous-mêmes sur quel type d'appareil effectuer notre test.



Affichage de notre application peu importe la taille de notre écran

```
@Composable
fun AffirmationsApp() {
    // Récupère la direction de lecture actuelle (LTR = Left To Right, ou RTL = Right To Left).
    // Sert à calculer le padding correctement selon la langue (ex: arabe, hébreu utilisent RTL).
    val layoutDirection = LocalLayoutDirection.current

    Surface(
        // La Surface est un composant Material Design de base.
        // Elle sert de conteneur avec une couleur de fond, une élévation, ou une bordure.
        // Ici, elle joue le rôle de "fond d'écran" pour ton app.
        modifier = Modifier
            // fillMaxSize() : étend le composant pour occuper toute la largeur ET la hauteur.
            // Utile pour s'assurer que la Surface recouvre l'écran entier.
            .fillMaxSize()

            // statusBarPadding() : ajoute automatiquement un espace équivalent
            // à la hauteur de la barre de statut (en haut de l'écran).
            // Utile pour éviter que le contenu soit masqué derrière l'heure, la batterie, etc.
            .statusBarPadding()

            // padding() : ajoute de l'espace autour du contenu (marges internes).
            // Ici, on l'utilise avec WindowInsets pour gérer les "zones sûres".
            .padding(
                // safeDrawing : représente les "zones sûres" de l'écran (hors encoches, bords arrondis, etc.).
                // asPaddingValues() : convertit ça en valeurs de padding utilisables.
                // calculateStartPadding(layoutDirection) : calcule le padding à gauche (ou à droite en RTL).
                start = WindowInsets.safeDrawing.asPaddingValues()
                    .calculateStartPadding(layoutDirection),

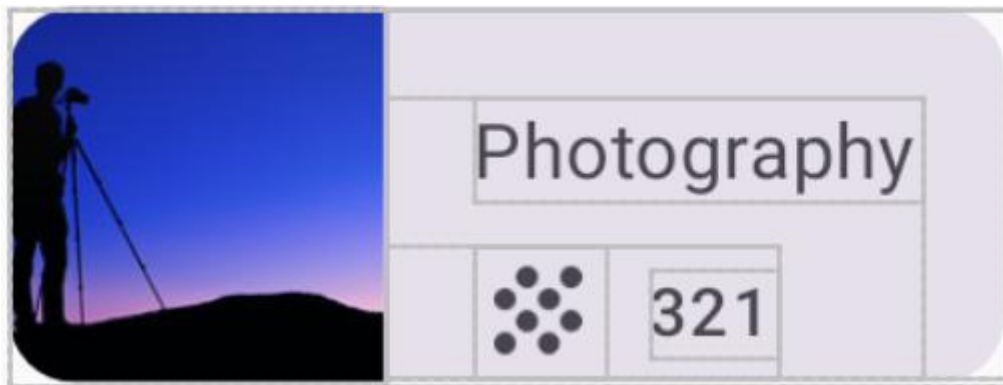
                // calculateEndPadding(layoutDirection) : calcule le padding à droite (ou à gauche en RTL).
                end = WindowInsets.safeDrawing.asPaddingValues()
                    .calculateEndPadding(layoutDirection),
            ),
    ) {
        // Contenu de la Surface (actuellement vide).
        // Tout ce que tu mets ici héritera du padding et des marges définies plus haut.
    }
}
```

Construire une grille :

Utilisation de **Icon**.



Construire notre élément unitaire



On a une Card{Row{I,Column{Text,Row{Icon,Text}}}}

LazyGrids : LazyVerticalGrid (scroll Vertical) ou LazyHorizontalGrid (Scroll Horizontal)